

Презентация по разделу №3 дополнительного курса

Дисциплина: Архитектура компьютеров и операционные системы

Гусев Степан Андреевич

2026-05-06

Информация

Выполнение третьего раздела

Выводы

1. Информация

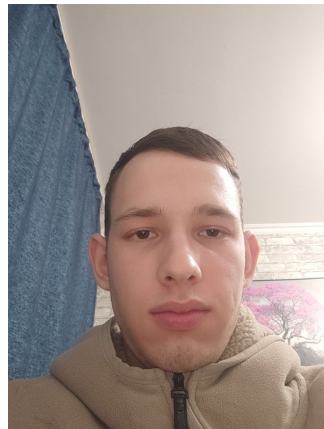
Презентация по разделу №3 дополнительного курса

Автор: Гусев Степан Андреевич

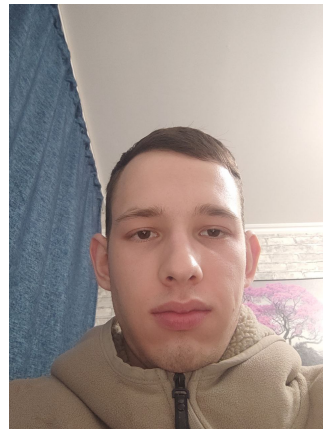
Преподаватель: Кулябов Дмитрий Сергеевич, д.ф.-м.н., профессор кафедры теории вероятностей и кибербезопасности

Российский университет дружбы народов

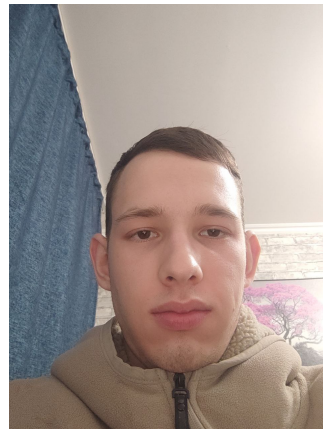
- Гусев Степан Андреевич



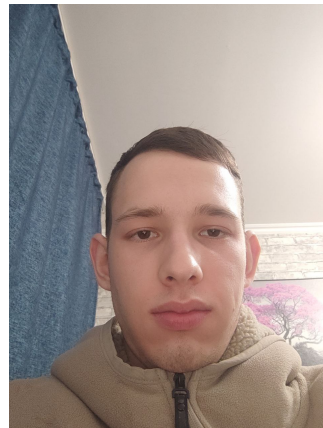
- Гусев Степан Андреевич
- Студент программы «Бизнес-информатика»



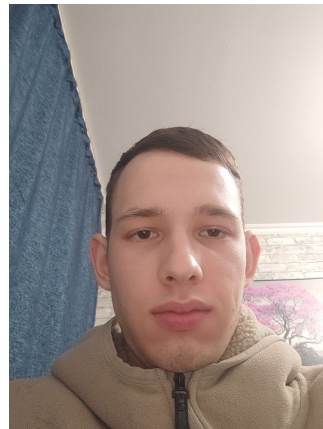
- Гусев Степан Андреевич
- Студент программы «Бизнес-информатика»
- Российский университет дружбы народов им. П. Лумумбы



- Гусев Степан Андреевич
- Студент программы «Бизнес-информатика»
- Российский университет дружбы народов им. П. Лумумбы
- 1032242444@rudn.ru



- Гусев Степан Андреевич
- Студент программы «Бизнес-информатика»
- Российский университет дружбы народов им. П. Лумумбы
- 1032242444@rudn.ru
- <https://github.com/stepagusev>



Приобретение практических навыков работы с Linux, пройти 3-ий раздел курса «Введение в Linux» на Stepik.

- 1) Пройти третий раздел курса «Введение в Linux»

2. Выполнение третьего раздела

Формулировка задания №1 (рис. 1).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Рисунок 1: Задание №1

Решение задания №1 (рис. 2).

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

☒ Прекрасный ответ.

☒ ":", затем "q", затем "Enter"

☐ "Q"

☐ "Ctrl", затем "x"

☐ "Esc"

☐ ":", затем "q"

Следующий шаг

Решить снова

Рисунок 2: Решение №1

Формулировка задания №2 (рис. 3).

При перемещении в vim "по словам" есть небольшая разница в том, используем мы маленькую (w, e, b) или большую (W, E, B) букву. Первые перемещают нас по "словам" (word), а вторые по "большим словам" (WORD). Посмотрите справку по этим перемещениям и разберитесь в чем заключается разница между word и WORD.

А для того, чтобы убедиться, что вы разобрались, отметьте ниже **все верные** утверждения про следующую строку:

```
Strange_ TEXT is_here. 2=2 YES!
```

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про word и WORD. Кроме того, можно вызвать сразу справку по термину word при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Рисунок 3: Задание №2

Задание 2

Решение задания №2: всего слов 9; чтобы попасть в конец, нужно меньше нажатий на «W», чем на «w» (рис. 4).

Выберите все подходящие ответы из списка

✓ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ В этой строке 5 "слов" (word)
- ☒ Нажимая только на W, нельзя переместить курсор на "."
- ☒ В этой строке 5 "больших слов" (WORD)
- ☐ В этой строке 9 "больших слов" (WORD)
- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
- ☐ В этой строке 12 "слов" (word)

Следующий шаг Решить снова

Рисунок 4: Решение №2

Формулировка задания №3 (рис. 5).

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже **наборов нажатий клавиш** выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Рисунок 5: Задание №3

Решение задания №3 (рис. 6).

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ d2wwywPp
- ☒ xxxxxxxxwywPp
- ☒ d2wwifour four <Esc>
- ☒ d2w\$bifour four <Esc>
- ☒ ddithree four four four five<Esc>
- ☐ d2wwywpp

Следующий шаг

Решить снова

Рисунок 6: Решение №3

Формулировка задания №4 (рис. 7).

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Рисунок 7: Задание №4

Решение задания №4 (рис. 8).

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Отлично!

:%s/Windows/Linux/

Следующий шаг

Решить снова

Рисунок 8: Решение №4

Формулировка задания №5 (рис. 9).

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Рисунок 9: Задание №5

Решение задания №5, для выхода из режима выделения «Esc» (рис. 10).

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☒ Режим выделения открывается из нормального режима по нажатию "v"
- ☒ В режиме выделения можно использовать команды d (удалить) и у (скопировать)

Рисунок 10: Решение №5

Формулировка задания №6 (рис. 11).

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Рисунок 11: Задание №6

Задание 6

Решение задания №6. При смене оболочки, история прошлой стирается (рис. 12).

Выберите один вариант из списка

☒ Хорошая работа.

- ☐ Никакие команды появляться не будут
- ☐ Из наборов В и С
- ☐ Только из набора А
- ☐ Только из набора В
- ☒ Только из набора С

Следующий шаг

Решить снова

Рисунок 12: Решение №6

Формулировка задания №7 (рис. 13).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Рисунок 13: Задание №7

Задание 7

Решение задания №7, файл был создан в /home/bi, последующий переход в другую директорию не влияет ни на то (рис. 14).

Выберите один вариант из списка



Абсолютно точно.

- ☐ /home/bi/Desktop/file1.txt
- ☒ /home/bi/file1.txt
- ☐ /home/bi/Documents/file1.txt
- ☐ Никак (файла file1.txt не будет существовать после завершения работы скрипта)

Следующий шаг

Решить снова

Рисунок 14: Решение №7

Формулировка задания №8 (рис. 15).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Рисунок 15: Задание №8

Задание 8

Решение задания №8: в названии могут быть только латинские буквы, цифры и нижнее подчёркивание, название переменной начинается с буквы (рис. 16).

Выберите все подходящие ответы из списка



Абсолютно точно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся, разместив свое решение с другими на [форуме решений](#).

- ☒ variable123
- ☒ VARiable
- ☐ var.i.able
- ☐ var-i-able
- ☒ variable_123
- ☐ vari/able
- ☒ Презентация

Формулировка задания №9 (рис. 17).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [arguments.sh](#).

Напишите скрипт на bash, который принимает на вход два аргумента и выводит на экран строку следующего вида:

```
Arguments are: $1=первый_аргумент $2=второй_аргумент
```

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Рисунок 17: Задание №9

Задание 9

Решение задания №9 (рис. 18).

Напишите программу. Тестируется через `stdin` → `stdout`

✓ Правильно, молодец!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить

```
1
2 echo "Arguments are: \$1=$1 \$2=$2"
3
4
5
6
```

Следующий шаг

Решить снова

Рисунок 18: Решение №9

Формулировка задания №10 (рис. 19).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: [branching1.sh](#).

Предположим, вы пишете скрипт на bash и хотите использовать в нем конструкцию `if` в следующем фрагменте:

```
if [[ ... ]]
then
  echo "True"
fi
```

Вы можете вписать вместо `"..."` (внутри `[[ ]]` и **не забудьте про пробелы** после `[[` и перед `]]`!) любое из перечисленных ниже условий. Однако мы просим вас выбрать только те из них, при которых `echo` напечатает на экран `True` вне зависимости от того, с какими параметрами был запущен ваш скрипт и какие в нем есть переменные.

Например, условие `0 -eq 0` **подходит**, т.к. ноль всегда равен нулю вне зависимости от аргументов и переменных внутри скрипта и на экран будет напечатано `True`. В то же время условие `$var1 -eq 0` **не подходит**, так как в переменной `var1` как может быть записан ноль (тогда будет напечатано `True`), так его может и не быть (тогда ничего напечатано не будет).

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Рисунок 19: Задание №10

Задание 10

Решение задания №10 (рис. 20).

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментарии](#) к решению с другими на [форуме решений](#).

- ☒ \$# -ge 0
- ☐ \$# -gt 0
- ☒ -s \$0
- ☒ \$var1 == \$var2 || \$var1 != \$var2
- ☒ ! (4 -le 3)
- ☐ -z " "

Следующий шаг

Решить снова

Рисунок 20. Решение №10

Формулировка задания №11 (рис. 21).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [branching2.sh](#), [branching3.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную **var=3**, а затем запустили еще раз, но уже с **var=5**.

Рисунок 21: Задание №11

Задание 11

Решение задания №11 (рис. 22).

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ Сначала two, потом one
- ☒ Сначала four, потом four
- ☐ Сначала four, потом one
- ☐ Сначала one, потом two

Следующий шаг

Решить снова

Рисунок 22: Решение №11

Формулировка задания №12 (рис. 23).

Напишите скрипт на bash, который принимает на вход один аргумент (целое число от 0 до бесконечности), который будет обозначать число студентов в аудитории. В зависимости от значения числа нужно вывести разные сообщения.

Соответствие входа и выхода должно быть таким:

```
0 --> No students
1 --> 1 student
2 --> 2 students
3 --> 3 students
4 --> 4 students
5 и больше --> A lot of students
```

Примечание а): выводить нужно только строку справа, т.е. "-->" выводить не нужно.

Примечание б): в последней строке слово **"lot"** с маленькой буквы!

Примечание 2: в этой и всех последующих задачах на написание скриптов, если не указано явно, что нужно **проверять вход** (например, что он будет именно числом и именно от 0 до бесконечности), то этого делать **не нужно!**

Пример №1: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 1` на экране должно появиться:

```
1 student
```



Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Рисунок 23: Задание №12

Задание 12

Решение задания №12 (рис. 24).

Напишите программу. Тестируется через stdin → stdout



Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 count=$1
2
3 if [ "$count" -eq 0 ]; then
4     echo "No students"
5 elif [ "$count" -eq 1 ]; then
6     echo "1 student"
7 elif [ "$count" -ge 2 ] && [ "$count" -le 4 ]; then
8     echo "$count students"
9 else
10    echo "A lot of students"
11 fi
```

Следующий шаг

Решить снова

Формулировка задания №13 (рис. 25).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент bash-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Рисунок 25: Задание №13

Решение задания №13 (рис. 26).

Выберите один вариант из списка



Правильно, молодец!

- ☐ 3 раза "start" и ни разу "finish"
- ☐ 5 раз "start" и 2 раза "finish"
- ☐ 5 раз "start" и ни разу "finish"
- ☒ 5 раз "start" и 4 раза "finish"

Следующий шаг

Решить снова

Формулировка задания №14 (рис. 27).

Напишите скрипт на bash, который будет определять в какую возрастную группу попадают пользователи. При запуске скрипт должен вывести сообщение **"enter your name:"** и ждать от пользователя ввода имени (используйте `read`, чтобы прочесть его). Когда имя введено, то скрипт должен написать **"enter your age:"** и ждать ввода возраста (опять нужен `read`). Когда возраст введен, скрипт пишет на экран **"<Имя>, your group is <группа>"**, где **<группа>** определяется на основе возраста по следующим правилам:

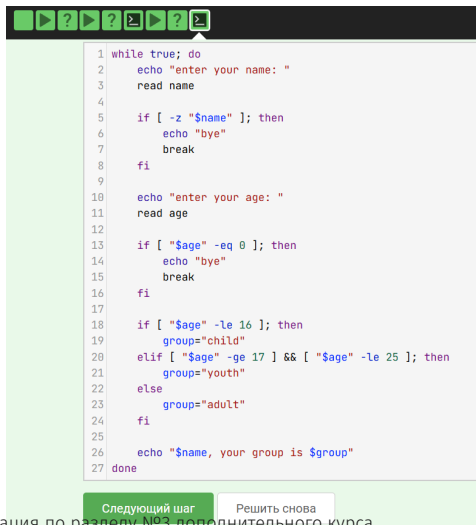
- младше либо равно 16: **"child"**,
- от 17 до 25 (включительно): **"youth"**,
- старше 25: **"adult"**.

После этого скрипт опять выводит сообщение **"enter your name:"** и всё начинается по новой (бесконечный цикл!). Если в какой-то момент работы скрипта будет введено **пустое имя** или **возраст 0**, то скрипт должен написать на экран **"bye"** и закончить свою работу (выход из цикла!).

Рисунок 27: Задание №14

Задание 14

Решение задания №14 (рис. 28).



```
1 while true; do
2     echo "enter your name: "
3     read name
4
5     if [ -z "$name" ]; then
6         echo "bye"
7         break
8     fi
9
10    echo "enter your age: "
11    read age
12
13    if [ "$age" -eq 0 ]; then
14        echo "bye"
15        break
16    fi
17
18    if [ "$age" -le 16 ]; then
19        group="child"
20    elif [ "$age" -ge 17 ] && [ "$age" -le 25 ]; then
21        group="youth"
22    else
23        group="adult"
24    fi
25
26    echo "$name, your group is $group"
27 done
```

Следующий шаг Решить снова

Формулировка задания №15 (рис. 29).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.

Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Рисунок 29: Задание №15

Решение задания №15 (рис. 30).

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ $a += b$
- ☒ `let a=a+b`
- ☒ `let "a = a + b"`
- ☐ $a = \$a + \b
- ☒ `let "a+=b"`

Следующий шаг

Решить снова

Рисунок 30: Решение №15

Формулировка задания №16 (рис. 31).

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: programs.sh.

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Рисунок 31: Задание №16

Решение задания №16, директория был изменена, echo выведет результат pwd (рис. 32).

Выберите один вариант из списка



Так точно!

☐ pwd

☐ `pwd`

☐ Код возврата команды pwd (0 в случае успешного выполнения и не 0 в случае ошибок)

☒ /home/bi

☐ /home/bi/Documents

Следующий шаг

Решить снова

Рисунок 32: Решение №16

Формулировка задания №17 (рис. 33).

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это **не всегда правда!** Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, используется именно **косая кавычка** (```), а не обычная (`"`) или двойная (`'`).

Рисунок 33: Задание №17

Задание 17

Решение задания №17 (рис. 34).

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [коротком](#) решении с другими на [форуме решений](#).

- ☐ Сначала var=`program`, затем if [[\$var -eq 0]]
- ☐ Ничего сделать нельзя
- ☒ if `program > some\_file.txt`
- ☒ Сначала запустить program, затем if [[\$? -eq 0]]
- ☐ if [[`program` -eq 0]]

Следующий шаг

Решить снова

Формулировка задания №18 (рис. 35).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
    local let "c1+=1"
    let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Рисунок 35: Задание №18

Решение задания №18, на месте с1 будет пустота, тк она локальная (рис. 36).

Напишите текст

✔ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Следующий шаг

Решить снова

Рисунок 36: Решение №18

Формулировка задания №19 (рис. 37).

Напишите скрипт на bash, который будет искать наибольший общий делитель ([НОД](#), greatest common divisor, GCD) двух чисел. При запуске ваш скрипт не должен ничего писать на экран, а просто ждет ввода двух натуральных чисел через пробел (для этого можно использовать `read` и указать ему две переменные – см. пример в видеофрагменте). После ввода чисел скрипт считает их НОД и выводит на экран сообщение "**GCD is <посчитанное значение>**", например, для чисел 15 и 25 это будет "GCD is 5". После этого скрипт опять входит в режим ожидания двух натуральных чисел. Если в какой-то момент работы пользователь ввел вместо этого пустую строку, то нужно написать на экран "**bye**" и закончить свою работу.

Вычисление НОД несложно реализовать с помощью [алгоритма Евклида](#). Вам нужно написать функцию `gcd`, которая принимает на вход два аргумента (назовем их **M** и **N**). Если **аргументы равны**, то мы нашли НОД – он равен **M** (или **N**), нужно вывести соответствующее сообщение на экран (см. выше). Иначе нужно сравнить аргументы между собой. Если **M больше N**, то запускаем ту же функцию `gcd`, но в качестве первого аргумента передаем (**M-N**), а в качестве второго **N**. Если же наоборот, **M меньше N**, то запускаем функцию `gcd` с первым аргументом **M**, а вторым (**N-M**).

Пример корректной работы скрипта:

```
./script.sh
10 15
GCD is 5
7 3
GCD is 1
bye
```

Рисунок 37: Задание №19

Задание 19

Решение задания №19 (рис. 38).

✓ Всё правильно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 gcd() {
2     local m=$1
3     local n=$2
4
5     if [ "$m" -eq "$n" ]; then
6         echo "GCD is $m"
7     elif [ "$m" -gt "$n" ]; then
8         gcd $((m - n)) $n
9     else
10        gcd $m $((n - m))
11    fi
12 }
13
14 while true; do
15     read -r a b
16     if [ -z "$a" ]; then
17         echo "bye"
18         break
19     fi
20     gcd "$a" "$b"
21 done
```

Следующий шаг

Решить снова

Рисунок 38: Решение №19

Формулировка задания №20 (рис. 39).

Напишите **калькулятор** на `bash`. При запуске ваш скрипт должен ожидать ввода пользователем команды (при этом на экран выводить ничего не нужно). Команды могут быть трех типов:

1. Слово **"exit"**. В этом случае скрипт должен вывести на экран слово **"bye"** и завершить работу.
2. **Три аргумента через пробел** – первый операнд (целое число), операция (одна из `"+"`, `"-"`, `"*"`, `"/"`, `"%"`, `"**"`) и второй операнд (целое число). В этом случае нужно произвести указанную операцию над заданными числами и вывести результат на экран. После этого переходим в режим ожидания новой команды.
3. **Любая другая команда** из одного аргумента или из трех аргументов, но с операцией не из списка. В этом случае нужно вывести на экран слово **"error"** и завершить работу.

Чтобы проверить работу скрипта, вы можете записать сразу несколько команд в файл и передать его скрипту на `stdin` (т.е. выполнить `./script.sh < input.txt`). В этом случае он должен вывести сразу все ответы на экран.

Например, если входной файл будет следующего содержания:

```
10 + 1
2 ** 10
exit
```

то на экране будет:

Рисунок 39: Задание №20

Решение задания №20 (рис. 40).

Теперь вам доступен [форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 while [[ True ]]
2 do
3   read fst amal snd
4   if [[ $fst == "exit" ]]
5   then
6     echo "bye"
7     break
8   elif [[ "$fst" =~ "^[0-9]+$" && "$snd" =~ "^[0-9]+$" ]]
9   then
10    echo "error"
11    break
12   else
13     case $amal in
14     "+") let "result = fst + snd";;
15     "-") let "result = fst - snd";;
16     "/" let "result = fst / snd";;
17     "*" let "result = fst * snd";;
18     "%" let "result = fst % snd";;
19     "**") let "result = fst ** snd";;
20     *) echo "error" ; break ;;
21     esac
22     echo "$result"
23   fi
24 done
```

Следующий шаг

Решить снова

Рисунок 40: Решение №20

Формулировка задания №21 (рис. 41).

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*" ?`

Рисунок 41: Задание №21

Задание 21

Решение задания №21, первая команда найдет файлы начинающиеся со «star» в любом регистре, вторая команда найдет файлы начинающиеся со «star» в нижнем регистре (рис. 42).

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*" ?`

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

- ☐ Eddard_Stark_biography.txt
- ☒ Star_Wars.avi
- ☒ STARS.txt
- ☐ stardust.mpeg
- ☐ star_trek OST.mp3

Следующий шаг

Решить снова

Рисунок 42: Решение №21

Формулировка задания №22 (рис. 43).

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Рисунок 43: Задание №22

Решение задания №22 (рис. 44).

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опция -path используется только для поиска директорий, а -name только для поиска файлов
- ☐ Если заменить в команде поиска -name, на -path, то результат поиска всегда останется неизменным
- ☐ Опции -path и -name всегда работают одинаково
- ☐ Опция -path аналогична -name, но игнорирует размер букв (строчные/прописные) в имени файла
- ☒ В некоторых случаях find с -name найдет меньше файлов, чем find с таким же запросом, но с -path

[Следующий шаг](#) [Решить снова](#)

Рисунок 44: Решение №22

Формулировка задания №23 (рис. 45).

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
├── dir1  
│   ├── file1  
│   ├── dir2  
│   │   ├── file2  
│   │   └── dir3  
│   │       └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Рисунок 45: Задание №23

Задание 23

Решение задания №23, $\text{depth}=1$ в текущей папке, mindepth и maxdepth работают включительно (рис. 46).

Выберите один вариант из списка

☒ Абсолютно точно.

☐ Ни один файл найден не будет

☐ Только file1

☒ Все кроме file3

☐ Только file2

☐ Все кроме file2

Следующий шаг

Решить снова

Рисунок 46: Решение №23

Формулировка задания №24 (рис. 47).

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в **каждой строке есть** слово `"word"`. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Рисунок 47: Задание №24

Решение задания №24 (рис. 48).

Выберите один вариант из списка



Абсолютно точно.

- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях

Следующий шаг

Решить снова

Рисунок 48: Решение №24

Формулировка задания №25 (рис. 49).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xk\lXKL]?[uü]buntu$" text.txt`.

Рисунок 49: Задание №25

Решение задания №25 (рис. 50).

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xkLXKL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

✓ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Well, xubuntu is OK
- ☐ Uuuubuntu!
- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04
- ☒ The best OS is Xubuntu
- ☐ Lubuntu is better than Windows
- ☐ Kbuntu

Следующий шаг

Решить снова

Рисунок 50: Решение №25

Формулировка задания №26 (рис. 51).

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n` ?

Рисунок 51: Задание №26

Решение задания №26 (рис. 52).

Что произойдет, если в команде `sed -n "[a-z]*p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Отлично!

- ☐ На экран ничего не напечатается
- ☐ Будут выведены все строки файла text.txt, в которых есть только большие буквы латинского алфавита
- ☐ На экран будет выведено всё содержимое файла text.txt
- ☒ Каждая строка будет выведена два раза

Следующий шаг

Решить снова

Рисунок 52: Решение №26

Формулировка задания №27 (рис. 53).

Запишите в форму ниже инструкцию `sed`, которая заменит все "аббревиатуры" в файле `input.txt` на слово "abbreviation" и запишет результат в файл `edited.txt` (на экран при этом ничего выводить не нужно). Обратите внимание, что **в инструкции должны быть** указаны и сам `sed`, и оба файла!

Под "аббревиатурой" будем понимать слово, которое удовлетворяет следующим условиям:

- состоит только из больших букв латинского алфавита,
- состоит из хотя бы двух букв,
- окружено одним пробелом с каждой стороны.

При этом будем считать, что в тексте **не может быть две "аббревиатуры" подряд**. Например, текст " YOU YOU and YOU!" является **некорректным** (в нем есть две "аббревиатуры", но они идут подряд) и на таких примерах мы проверять вашу инструкцию **не будем**.

Пример: если у вас был текст "Hi, I heard these songs by ABBA, TLA and DM!", то он должен быть преобразован в "Hi, I heard these songs by ABBA, abbreviation and abbreviation!".

Примечание: после вашей замены "аббревиатуры" на слово "abbreviation" **количество пробелов** в тексте **не должно меняться!**

Рисунок 53: Задание №27

Решение задания №27(рис. 54).

Напишите текст



Правильно.

```
sed -E 's/ ([A-Z]{2,}) / abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Рисунок 54: Решение №27

Формулировка задания №28 (рис. 55).

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Рисунок 55: Задание №28

Решение задания №28, регистр учитывается (рис. 56).

Вы можете скачать и попробовать применить `gnuplot` к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске `gnuplot`, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Всё правильно.

- ☐ Такой опции не существует
- ☒ `-p, -persist`
- ☐ Графики и так не закрываются автоматически при закрытии `gnuplot`!
- ☐ `-s, -show-plots-after-exit`

Следующий шаг

Решить снова

Рисунок 56: Решение №28

Формулировка задания №29 (рис. 57).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead  
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Рисунок 57: Задание №29

Решение задания №29 (рис. 58).

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название** у построенного **ряда данных** и **сколько** будет нарисовано **точек** на графике?

Выберите один вариант из списка

✓ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название – первое значение из второго столбца, нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название "data.csv" using 1:2, нарисовано 10 точек
- ☐ Название "no name", нарисовано 10 точек

Следующий шаг

Решить снова

Рисунок 58: Решение №29

Формулировка задания №30 (рис. 59).

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>".**

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Рисунок 59: Задание №30

Решение задания №30 (рис. 60).

Напишите текст



Прекрасный ответ.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Следующий шаг

Решить снова

Рисунок 60: Решение №30

Формулировка задания №31 (рис. 61).

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять** и **удалять** инструкции **нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка (10, 10, 200), станет точка (10, 10, -200), где была точка (-10, -10, 200) станет (-10, -10, -200) и т.д. При этом точка (0, 0, 0) останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Рисунок 61: Задание №31

Решение задания №31 (рис. 62).

Напишите текст



Хорошие новости, верно!

move.rot (88 bytes)

Следующий шаг

Решить снова

Рисунок 62: Решение №31

Формулировка задания №32 (рис. 63).

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrw-r--`, если изначально у него были права `r--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Рисунок 63: Задание №32

Решение задания №32 (рис. 64).

Какая команда(ы) установят файлу `file.txt` права доступа `rw-rw-r--`, если изначально у него были права `g--r--r--`. Укажите **все верные** варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod rw-rw-r-- file.txt`
- ☐ `chmod u-wx file.txt; chmod g-w file.txt`
- ☐ `chmod 777 file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☐ `chmod o-wx file.txt; chmod g-x file.txt; chmod a+wx file.txt`
- ☒ `chmod ug+w file.txt; chmod u+x file.txt`

Следующий шаг

Решить снова

Рисунок 64: Решение №32

Формулировка задания №33 (рис. 65).

3.7 Разное 5 из 15 шагов пройдено 2 из 7 баллов получено

Предположим вы использовали команду `sudo` для создания директории `dir`. По умолчанию для `dir` были выставлены права доступа `rw-r-xr-x` (владелец `root`, группа `root`). Таким образом никто кроме пользователя `root` не может ничего записывать в эту директорию, например, не может создавать файлы в ней.

После выполнения какой команды `user` из группы `group` всё-таки сможет создать файл внутри `dir`? Укажите **все верные** варианты ответов!

Примечание: считаем, что все команды выполняются от имени `user`, если явно не указано, что команда выполнена с `sudo`.

Примечание 2: мы выбрали пример с директорией, а не с файлом не случайно.

Дело в том, что если создать при помощи `sudo` файл с правами `rw-r--r--` в директории, которая принадлежит пользователю, то возникнет любопытная ситуация. С одной стороны пользователь может удалить этот файл (т.к. ему разрешено удалять **все** файлы внутри его директории) и может прочитать его содержимое (т.к. право `"r"` у файла установлено для всех), с другой стороны он не может этот файл редактировать (т.к. право `"w"` у файла есть только для `root`). При этом некоторые "умные" редакторы, например, `vim` позволят даже редактировать этот файл, но сделают они это своеобразно: через удаление оригинала и создание копии уже с нужными правами (удалять мы можем, а раз можем читать, то и копию создать не сложно). Итого получается, что несмотря на права `rw-r--r--`, пользователь может сделать с этим файлом почти всё что угодно!

В случае же, когда речь идет о директории созданной `root`, ситуация будет проще: пользователь сможет посмотреть её содержимое (у него есть право `"r"`), но удалять и создавать файлы в ней не сможет (права `"w"` у него нет).

Важно отметить, что директории в `Linux` это в каком-то смысле *файлы*. Содержимое такого "файла" – это записи о файлах и поддиректориях этой директории (грубо говоря их названия). Таким образом, право `"r"` у директории дает возможность просматривать "записи". т.е. просматривать её состав. Право `"w"` у директории дает возможность удалять/добавлять новые "записи".

Рисунок 65: Задание №33

Решение задания №33 (рис. 66).

Выберите все подходящие ответы из списка

✓ Верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их решение с другими на [форуме решений](#).

- ☐ sudo chmod g+w dir
- ☒ sudo chown user:group dir
- ☒ sudo chown user dir
- ☐ sudo chown :group dir
- ☒ sudo chmod o+w dir
- ☐ sudo chmod o+x dir

Следующий шаг

Решить снова

Рисунок 66: Решение №33

Формулировка задания №34 (рис. 67).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Рисунок 67: Задание №34

Решение задания №34 (рис. 68).

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их решение с другими на [форуме решений](#).

- ☒ Размер файла в байтах
- ☒ Длину самой длинной строки
- ☐ Количество предложений
- ☐ Количество определенных букв (например, количество букв "А")
- ☒ Количество символов

Следующий шаг

Решить снова

Рисунок 68: Решение №34

Формулировка задания №35 (рис. 69).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо `2848 байт` надо выводить `2.8K`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4M` (также на экране может быть выведен еще и символ `.`, обозначающий, что это размер именно текущей директории).

Рисунок 69: Задание №35

Решение задания №35 (рис. 70).

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести в **удобном для чтения формате** (например, вместо `2048 байт` надо вывести `2.0K`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4M` (также на экране может быть выведен еще и символ `..`, обозначающий, что это размер именно текущей директории).

Напишите текст

✓ Всё получилось!

```
du -sh .
```

Следующий шаг

Решить снова

Вы получили: 2 балла

Верно решили 25 135 учащихся

Рисунок 70: Решение №35

Формулировка задания №36 (рис. 71).

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Рисунок 71: Задание №36

Решение задания №36 (рис. 72).

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст

✓ Здорово, всё верно.

```
mkdir dir{1,2,3}
```

Следующий шаг

Решить снова

Рисунок 72: Решение №36

3. Выводы

В процессе проделанной работы я приобрёл практические навыки работы с Linux и прошёл 3-ий раздел курса «Введение в Linux» на Stepik.